# SmartUSBHub Python SDK

## API Reference

Applies to: SmartUSBHub Product Family (SDK v1.3.1)

Released · Document updated: November 12, 2026

www.mixedsignallab.com

# Contents

Introduction . . . . . 2
Product Matrix . . . . . 2
Product Overview . . . . . 2
Connection . . . . . 3
CMD Control Port . . . . . 3
Quick Start . . . . . 3
Method 1: Install the release package . . . . . 3
Method 2: Use this source repository . . . . . 3
Running the examples . . . . . 3
Integrating into your project . . . . . 4
API Reference . . . . . 4
Connection Management . . . . . 4
Channel Control . . . . . 6
Measurement and Monitoring . . . . . 8
Alerts and Protection . . . . . 10
Default States . . . . . 12
Device Configuration . . . . . 15
Device Information and Discovery . . . . . 17
Event Callbacks . . . . . 20

Documentation map: Quick Start · Communication Protocol · Product Documentation · Examples Guide

Documentation version: v1.3.1 · Last updated: November 12, 2026

SDK 1.3.1 fixes compatibility with older V3 measurement data and streaming-control acknowledgements. Public APIs and the protocol API version are unchanged; see the changelog for details.

This documentation applies to SmartUSBHub models supported by the SDK. Before use, confirm your device model and select the corresponding product documentation.

# Introduction

The SmartUSBHub Python library is used to control SmartUSBHub devices in scripts, test systems, and automation workflows. To control the device directly with software, use the companion control software shipped with the release package, available at the download page.

SmartUSBHub models differ in port count, power capability, measurement capability, and dedicated features. Select the corresponding product documentation for your model before use.

## Product Matrix

| Product | Model / Order Code | Channels | USB Spec | Datasheet | User Guide |
|---|---|---|---|---|---|
| SmartUSBHub Pro 4CH USB2.0 | HBP_USB2_4CH, HBP_USB2_4CH_PSU | 4 | USB 2.0 High-Speed | Datasheet | User Guide |
| SmartUSBHub Pro 7CH USB2.0 | HBP_USB2_7CH, HBP_USB2_7CH_ADV | 7 | USB 2.0 High-Speed | Datasheet | User Guide |
| SmartUSBHub Pro 4CH USB3.0 | HBP_USB3_4CH_ADV | 4 | USB 3.0 Super Speed | — | — |

For other supported order codes, refer to the model-specific documentation shipped with the device.

# Product Overview

SmartUSBHub is a family of programmable USB hubs with per-channel power control. Some models also support data-line control, and some support voltage/current monitoring, making them suitable for development, automated testing, and device management.

Each downstream port has independent control of power (VBUS) and the data connection: USB 2.0 models switch D+ / D−, and models that support USB3.0 can additionally switch SuperSpeed data lines independently. USB devices can be connected and disconnected remotely without manual plugging, for automated testing, remote restarts, and hot-plug simulation. Models with voltage/current monitoring sample each channel's voltage and current in real time for power-draw analysis and supply-state monitoring; abnormal voltage drops or overcurrent during testing can be detected and located promptly.

Each port supports a power-on default state and power-off state restoration; both normal mode (multiple channels controlled simultaneously) and interlock mode (only one channel operable at a time) are supported. Multiple devices can be managed in a cascaded topology under centralized control, and each device can be assigned a device address. The control port uses USB CDC and is

driver-free, making it easy to integrate port control into automated test platforms, Python scripts, orother tools.

# Connection

> NOTE: Connect the device's DATA port to the USB host with a USB data cable
>
> The DATA port is the upstream port for USB data: downstream USB devices reach the host through it, and the operating system recognizes a standard USB hub.
>
> When one host must both control the SmartUSBHub and communicate with downstream USB devices, both DATA and CMD must be connected.

## CMD Control Port

Connect the device's CMD control port to the control computer with a USB data cable. The CMD port uses USB CDC and is driver-free; once connected it enumerates as a standard serial port, COMx on Windows, /dev/ttyACMx on Linux, and /dev/cu.usbmodemx on macOS.

The CMD port's USB descriptors are VID 0x1A86, PID 0xFE0C, with iSerialNumber set to the device serial number. scan_available_ports() filters control ports by VID/PID; when multiple devices are connected to one host, they can be distinguished by serial number.

# Quick Start

Choose either usage method.

## Method 1: Install the release package

```
pip install smartusbhub
```

## Method 2: Use this source repository

```
cd smartusbhub
python -m venv venv
source ./venv/bin/activate
python -m pip install -r requirements.txt
python -m pip install -e .
```

The repository layout is:

```
.
├── README.md          # Documentation
├── docs               # Product guides and communication protocol
├── examples           # Example scripts
├── requirements.txt   # SDK core dependencies
└── smartusbhub.py     # Library source
```

## Running the examples

Examples for the smartusbhub library live in the examples directory; running them in order is the fastest way to bring up a device:

- quick_start.py: prints a full device report (identity, capabilities, runtime status), then demonstrates power control: single channel, all channels on/off together, interlock mode, and device address read/write
- power_measurement_oneshot.py: reads each channel's voltage and current continuously via fixed-length protocol request/response on models with measurement support
- multi_device_channel_control.py: discovers and controls multiple devices

See the examples guide for details.

Run the examples from the repository root, for example:

```
python examples/quick_start.py
python examples/power_measurement_oneshot.py
```

## Integrating into your project

Import the smartusbhub library to integrate it into your project.

The SDK is licensed under Apache-2.0. When integrating or redistributing, follow Article 4 of Apache-2.0 and retain the license, copyright notices, and attribution notices in NOTICE in a readable form.

1. Import the smartusbhub library into your project:

```
from smartusbhub import SmartUSBHub
```

2. Initialize a SmartUSBHub instance. For automatic scan:

```
hub = SmartUSBHub.scan_and_connect()
```

Or connect by serial port path (for example "/dev/cu.usbmodem132301"):

```
hub = SmartUSBHub("/dev/cu.usbmodem132301")
```

# API Reference

## Connection Management

### scan_and_connect(exclude_ports=None, device_address=None)

Description: Scans matching USB CDC serial ports and connects to the first available device.

Parameters: exclude_ports (list[str]): set of ports to skip; ports that are already connected are skipped by default. device_address (int): only connect to a device reporting this address. Device addresses default to zero, multiple devices may share one address, and addresses are not unique until configured; prefer selecting by port.

Returns: SmartUSBHub or None: the connected instance; None when no device is available.

Example:

```
hub = SmartUSBHub.scan_and_connect()
```

### scan_available_ports()

Description: Returns serial port paths whose USB VID/PID match SmartUSBHub.

Returns: list[str].

Example:

```
ports = SmartUSBHub.scan_available_ports()
```

### auto_connect(exclude_ports=None, feature_filter=None)

Description: Tries candidate ports in turn, automatically skipping ports that are occupied or fail to connect.

Parameters: exclude_ports (list[str]): set of ports to skip; ports that are already connected are skipped by default. feature_filter (str): only connect to a device supporting this feature; one of adc, usb2_data_switch, usb3_data_switch, or ilim_switch.

Returns: SmartUSBHub or None.

Example:

```
hub = SmartUSBHub.auto_connect(feature_filter="adc")
```

### SmartUSBHub(port)

Description: Opens the specified SmartUSBHub serial port.

Parameters: port (str): device serial port path, e.g. "/dev/cu.usbmodem132301".

Example:

```
hub = SmartUSBHub("/dev/cu.usbmodem132301")
```

### disconnect()

Description: Stops the receiving thread, closes the serial port, and releases the process lock. Safe to call repeatedly.

Example:

```
hub.disconnect()
```

### is_connected()

Description: Returns whether the serial port is currently open.

Returns: bool.

### close()

Description: Alias of disconnect().

Returns: None.

You can also manage the connection with a with statement; the device disconnects automatically when the with block exits:

Example:

```
with SmartUSBHub("/dev/cu.usbmodem132301") as hub:
    hub.set_channel_power(1, state=1)
```

## Channel Control

### set_channel_power(*channels, state)

Description: Sets the power state of the specified channels.

Parameters: *channels (int): channels to control. state (int): 1 turns power on, 0 turns power off.

Returns: bool: True if the command was applied successfully, otherwise False.

Example:

```
hub.set_channel_power(1, 2, state=1)
```

### get_channel_power_status(*channels)

Description: Queries the power state of the specified channels.

Parameters: *channels (int): channels to query, as variadic arguments; available channels can be obtained via hub.get_channels().

Returns: dict or int or None: a dict of channel states when querying multiple channels; the channel's state when querying a single channel; None on timeout.

Example:

```
status = hub.get_channel_power_status(1, 2)
```

### set_channel_usb2_dataline(*channels, state)

Description: Sets the USB data switch state of the specified channels.

Parameters: *channels (int): channels to update, as variadic arguments; available channels can be obtained via hub.get_channels(). state (int): 1 connects D+ / D−, 0 disconnects D+ / D−.

Returns: bool: True if the command was applied successfully, otherwise False.

Example: Connect the data signals of channel 1:

```
hub.set_channel_usb2_dataline(1,state=1)
```

### get_channel_usb2_dataline_status(*channels)

Description: Queries the USB data switch state of the specified channels.

Parameters: *channels (int): channels to query, as variadic arguments; available channels can be obtained via hub.get_channels().

Returns: dict or None: a dict of channel states; None on timeout.

Example: Get the USB 2.0 data connection state of channels 1 and 2:

```
status = hub.get_channel_usb2_dataline_status(1, 2)
```

### set_channel_usb3_dataline(*channels, state)



NOTE
Note: This API applies only to models that declare the USB3.0 data switch capability (e.g. HBP_USB3_4CH_ADV). Unsupported devices raise FeatureNotSupportedError.

Description: Sets the USB 3.0 data switch state of the specified channels.

Parameters: *channels (int): channels to update, as variadic arguments; available channels can be obtained via hub.get_channels(). state (int): 1 connects, 0 disconnects.

Returns: bool: True when the device acknowledges, False on timeout.

Example:

```
hub.set_channel_usb3_dataline(1, state=1)
```

### get_channel_usb3_dataline_status(*channels)

Description: Queries the USB 3.0 data switch state of the specified channels; all valid channels when omitted.

Parameters: *channels (int): channels to query.

Returns: dict or None: {channel: state}; None on timeout.

Example:

```
status = hub.get_channel_usb3_dataline_status(1, 2)
```

### set_channel_power_interlock(channel)

Description: Selects the single powered channel in interlock mode, or turns all channels off. When a channel's power is turned off, its data connection is disconnected as well.

Note: in interlock mode the normal power command set_channel_power() has no effect; use set_channel_power_interlock() to select the powered channel. The interlock command itself controls power; to control the selected channel's USB 2.0 data connection, use set_channel_usb2_dataline().

Parameters: channel (int or None): the channel to set. If None, all channels are turned off.

Returns: bool: True if the command was applied successfully, otherwise False.

Example:

```
hub.set_channel_power_interlock(1)
```

### set_upstream_host(host)



NOTE
Note: This API applies only to models that declare the upstream_host_switch capability (dual-host switching). Unsupported devices raise FeatureNotSupportedError.

Description: Selects the upstream host the device is connected to. The selection is stored on the device and persists across power cycles.

Parameters: host (int): upstream host number, 1 or 2.

Returns: bool: True when a matching confirmation is received; False when rejected by the device or on timeout.

Example: Switch to host 2:

```
hub.set_upstream_host(2)
```

### get_upstream_host()

Description: Queries the upstream host currently used by the device.

Returns: int or None: 0 (unknown), 1 (host 1), or 2 (host 2); None when rejected or on communication failure.

Example:

```
host = hub.get_upstream_host()
```

## Measurement and Monitoring

### get_channel_voltage(channel)



NOTE
Note: This API applies only to models with voltage and current monitoring.

Description: Queries the voltage of a single channel.

Parameters: channel (int): the channel to query.

Returns: int or None: the channel's voltage (mV); None on timeout.

Example: Get the voltage of channel 1:

```
voltage = hub.get_channel_voltage(1)
```

### get_channel_current(channel)



NOTE
Note: This API applies only to models with voltage and current monitoring.

Description: Queries the current of a single channel.

Parameters: channel (int): the channel to query.

Returns: int or None: the channel's current (mA); None on timeout.

Example: Get the current of channel 1:

```
current = hub.get_channel_current(1)
```

### get_channel_measurements(*channels)

> NOTE
> Note: This API applies only to models with voltage and current monitoring.

Description: Reads voltage and current measurements of one or more channels in a single request. All valid channels of the current product are read when no channel is specified.

Parameters: *channels (int): channels to query, as variadic arguments; available channels can be obtained via hub.get_channels().

Returns: dict or None: returns {channel: {"voltage": mV, "current": mA, "fresh": bool, "stale": bool, "valid": bool}}; None on timeout or when no valid data is available.

Example:

```
measurements = hub.get_channel_measurements(1, 2)
all_measurements = hub.get_channel_measurements(*hub.get_channels())
```

### set_channel_measurement_stream(*channels, enabled=True, wait_ack=True)

> NOTE
> Note: This API applies only to models that support continuous measurement output.

Description: Enables or disables continuous measurement output for the specified channels. All valid channels of the current product are targeted when no channel is specified.

Parameters: *channels (int): channels to stream measurements for. enabled (bool): True to enable, False to disable. wait_ack (bool): whether to wait for the device acknowledgment.

Returns: bool: True on success, False on timeout.

Example:

```
hub.set_channel_measurement_stream(*hub.get_channels(), enabled=True)
```

### get_stream_channel_measurements(*channels, timeout=None, wait_new_sample=True)

Description: Waits for the next continuous-measurement frame and returns measurements for the specified channels. Continuous measurement output must be enabled first.

Parameters: *channels (int): channels to read. timeout (float): wait timeout in seconds; the default communication timeout is used when omitted. wait_new_sample (bool): True waits for a new sample, False accepts the next frame.

Returns: dict or None: a dict containing voltage, current, sample_tick, sample_period_ms, and related fields; None on timeout.

Note: batch capture is consumed in order and does not keep only the last frame of a batch. sample_time_us is the time the MCU issued the REFRESH request (wrapping 32-bit microseconds), not the exact ADC conversion instant; capture_statistics and measurement_queue_drops report sample loss on the device side and host side respectively. Model-specific sample-rate tiers and filtering characteristics are documented in the corresponding product documentation.

Example:

```
data = hub.get_stream_channel_measurements(1, 2, timeout=1.0)
```

### get_cached_measurements(*channels)

Description: Reads the latest measurements cached by the background receiving thread without blocking for new data. Typically used together with continuous measurement output.

Parameters: *channels (int): channels to read.

Returns: dict or None: the latest cached measurements; None if no measurement data has been received yet.

Example:

```
latest = hub.get_cached_measurements(*hub.get_channels())
```

### get_measurement_sample_rate() and set_measurement_sample_rate(rate)

Description: Reads or sets the measurement sample rate declared by the device (in Sa/s). Ongoing capture is paused and resumed during the change; setting an undeclared rate raises FeatureNotSupportedError.

Parameters: rate (int): target sample rate in Sa/s; must be a rate declared by the device.

Returns: the getter returns int or None; the setter returns bool.

Related: supported_sample_rates() returns the tuple of sample rates declared by the device; supports_measurement_capture() reports whether the device supports batch capture.

Example:

```
rates = hub.supported_sample_rates()
if 100 in rates:
    hub.set_measurement_sample_rate(100)
```

### get_measurement_filter() and set_measurement_filter(mode)

Description: Reads or selects the measurement hardware filter mode. Raises FeatureNotSupportedError when the device does not declare a configurable filter. The setting is kept in RAM only and defaults to the instantaneous value after restart; switching briefly stops the stream and clears old batches.

Parameters: mode (int): 0 for the instantaneous value; 1 for the hardware's built-in eight-conversion sliding average, with the output sample rate unchanged.

Returns: the getter returns int (0/1) or None; the setter returns bool.

## Alerts and Protection

### get_channel_oc_status()



NOTE
Note: Both HBP_USB2_7CH and HBP_USB2_7CH_ADV support per-port overcurrent status. Availability on other models depends on hardware and firmware.

Description: Queries the real-time and latched overcurrent state of every channel.

Returns: dict or None: returns {channel: {"active": bool, "latch": bool}}. active is the current overcurrent state and latch indicates a latched overcurrent event. None on timeout.

Example:

```
oc_status = hub.get_channel_oc_status()
```

### clear_channel_oc_latch(*channels)

Description: Clears the overcurrent latch of the specified channels. All channels are cleared when no channel is specified.

Parameters: *channels (int): channels whose latch should be cleared.

Returns: bool: True on success, False on timeout.

Example:

```
hub.clear_channel_oc_latch(1, 2)
hub.clear_channel_oc_latch()
```



NOTE
Note: This group of APIs applies only to models that declare the alerts capability (e.g. HBP_USB3_4CH_ADV). Unsupported devices raise FeatureNotSupportedError.

Threshold rules support five kinds — overcurrent, undercurrent, overvoltage, undervoltage, and overpower — with thresholds in mA, mA, mV, mV, and mW respectively. Comparator thresholds are quantized according to the device declaration; layout and limits are documented in the communication protocol.

Rules are disabled by default and kept in RAM; before set_alert_config() returns successfully, the rule is written to the application Flash (identical configurations are not rewritten). action may be notify (report only) or power_off_lock (cut power and lock: disconnect power and data, and block all power-on paths until explicitly unlocked).

### set_alert_config(channel, kind, threshold, enabled=True, samples=1, action="notify")

Description: Sets one threshold rule.

Parameters: channel (int): channel the rule belongs to. kind (str): alert type; see the five types above. threshold (int): threshold value; unit is mA, mV, or mW depending on the type. enabled (bool): whether the rule is enabled. samples (int): number of consecutive comparator hits; choose from 1/4/8/16. action (str): notify (report only) or power_off_lock (cut power and lock).

Returns: dict or None: the rule dict actually applied by the device, or None on failure.

### get_alert_config(channel, kind) and get_alert_defaults(channel, kind)

Description: Reads the currently effective rule and the MCU factory-default rule respectively, without changing existing configuration.

Parameters: channel (int): channel the rule belongs to. kind (str): alert type; see the five types above.

Returns: dict or None.

### get_alert_status()

Description: Reads the cached alert status, including ready, io_fault, event sequence numbers, dropped event counts, and per-channel active (current alerts), history (alert history), trip (protection lock), and power (power state). Does not consume the chip's read-to-clear registers.

Returns: dict or None.

### subscribe_alert_events(enabled=True)

Description: Subscribes to alert events proactively reported by the device. A CMD_ALERT_EVENT callback must be registered via register_callback() first; the callback runs in the receiving thread.

Parameters: enabled (bool): True to subscribe, False to unsubscribe.

Returns: bool.

### clear_alert_latch(channel_mask) and reset_protection_trip(channel_mask)

Description: Clears alert history or unlocks protection. clear_alert_latch() clears only alert history and does not unlock protection; reset_protection_trip() unlocks ports that were cut off once the monitored readings recover — it does not power them back on automatically, re-powering is an explicit action, and protection triggers again if the load fault persists.

Parameters: channel_mask (int): channel bitmask; bit 0 corresponds to channel 1.

Returns: bool.

Example:

```
hub.set_alert_config(1, "overcurrent", 2000, samples=8, action="power_off_lock")
status = hub.get_alert_status()
# After fault handling, the port remains powered off; once the alert clears and monitoring is healthy:
hub.clear_alert_latch(0x01)      # clear history only
hub.reset_protection_trip(0x01)  # unlock only, do not power on
```

## Default States

### set_default_power_status(*channels, enable, status=None)

Description: Sets the power-on default power state of the specified channels.

Parameters: *channels (int): channels to configure, as variadic arguments; available channels can be obtained via hub.get_channels(). enable (int or bool): 1/True enables the default state, 0/False disables it. status (int or bool, optional): 1/True defaults power on, 0/False defaults power off; defaults to off when omitted.

Returns: bool: True if the command was applied successfully, otherwise False.

Example: Channels 1–4 default power on:

```
hub.set_default_power_status(1, 2, 3, 4, enable=1, status=1)
```

Example: Channels 1–4 do not use a default on power-up:

```
hub.set_default_power_status(1, 2, 3, 4, enable=0)
```

### get_default_power_status(*channels)

Description: Queries the power-on default power state of one or more channels.

Parameters: *channels (int): channels to query, as variadic arguments; available channels can be obtained via hub.get_channels().

Returns: dict or None: {channel: {"enabled": enabled, "value": state}}, where enabled is 0 (disabled) or 1 (enabled), and value is 0 (default off) or 1 (default on). None on timeout.

Query the power-on default power state of channels 1–4 on a device whose defaults have never been configured:

Example:

```
hub.get_default_power_status(1, 2, 3, 4)
```

Returns:

```
{1: {'enabled': 0, 'value': 0}, 2: {'enabled': 0, 'value': 0}, 3: {'enabled': 0, 'value': 0}, 4: {'enabled': 0, 'value': 0}}
```

enabled being 0 means the channel does not use a power-on default, in which case value has no effect; only when enabled is 1 does value decide whether power is on or off at power-up.

### set_default_usb2_dataline_status(*channels, enable, status=None)

Description: Sets the power-on default data connection state of the specified channels.

Parameters: *channels (int): channels to configure, as variadic arguments; available channels can be obtained via hub.get_channels(). enable (int or bool): 1/True enables the default state, 0/False disables it. status (int or bool, optional): 1/True defaults data connected, 0/False defaults data disconnected; defaults to disconnected when omitted.

Returns: bool: True if the command was applied successfully, otherwise False.

Example: Channels 1–4 default to data connected on power-up:

```
hub.set_default_usb2_dataline_status(1,2,3,4,enable=1,status=1)
```

### get_default_usb2_dataline_status(*channels)

Description: Queries the power-on default data connection state of one or more channels.

Parameters: *channels (int): channels to query, as variadic arguments; available channels can be obtained via hub.get_channels().

Returns: dict or None: {channel: {"enabled": enabled, "value": state}}, where enabled is 0 (disabled) or 1 (enabled), and value is 0 (default disconnected) or 1 (default connected). None on timeout.

Get the power-on default data connection state of channels 1–4:

Example:

```
hub.get_default_usb2_dataline_status(1,2,3,4)
```

Returns:

```
{1: {'enabled': 0, 'value': 1}, 2: {'enabled': 0, 'value': 1}, 3: {'enabled': 0, 'value': 1}, 4: {'enabled': 0, 'value': 1}}
```

### set_default_usb3_dataline_status(*channels, enable, status=None)



NOTE
Note: This API applies only to models that declare the device.default_usb3_data capability (e.g. HBP_USB3_4CH_ADV). Unsupported devices raise FeatureNotSupportedError.

Description: Sets the power-on default state of the USB3.0 data lines for the specified channels and saves it to the device. A successful save does not change the current data connection immediately; it takes effect when the device restores state at power-up.

Parameters: *channels (int): channels to configure, as variadic arguments; available channels can be obtained via hub.get_channels(). enable (int or bool): 1/True enables the default state, 0/False disables it. status (int or bool, optional): 1/True defaults data connected, 0/False defaults data disconnected; defaults to disconnected when omitted.

Returns: bool: True when the firmware save and read-back verification succeed, otherwise False.

Note: with enable=0, the port falls back to its normal power-on default state. At power-up, a USB2.0 data default of disconnected and a power default of off take precedence over the USB3.0 default state.

Channels 1–2 default to USB3.0 data disconnected on power-up:

Example:

```
hub.set_default_usb3_dataline_status(1, 2, enable=1, status=0)
```

### get_default_usb3_dataline_status(*channels)



NOTE
Note: This API applies only to models that declare the device.default_usb3_data capability (e.g. HBP_USB3_4CH_ADV). Unsupported devices raise FeatureNotSupportedError.

Description: Queries the USB3.0 data power-on default state of one or more channels.

Parameters: *channels (int): channels to query, as variadic arguments; all channels when omitted.

Returns: dict or None: {channel: {"enabled": enabled, "value": state}}, with the same meaning as the USB2.0 default state. None on timeout.

Example: Get the USB3.0 data power-on default state of channels 1–2:

```
hub.get_default_usb3_dataline_status(1, 2)
```

### set_auto_restore(enable)

Description: Enables or disables power-off state restoration.

Parameters: enable (bool): True to enable, False to disable.

Returns: bool: True if the command was applied successfully, otherwise False.

Example: Enable power-off state restoration:

```
hub.set_auto_restore(True)
```

### get_auto_restore_status()

Description: Queries whether power-off state restoration is enabled.

Returns: int or None: 1 enabled, 0 disabled, None when the device does not respond.

Example: Query power-off state restoration:

```
status = hub.get_auto_restore_status()
```

## Device Configuration

### set_button_control(enable)

Description: Enables or disables the hub's physical button (when present).

Parameters: enable (bool): True enables the button, False disables it.

Returns: bool: True if the command was applied successfully, otherwise False.

Example: Enable the button:

```
hub.set_button_control(True)
```

### get_button_control_status()

Description: Queries whether the hub's physical button is enabled (when present).

Returns: int or None: 1 enabled, 0 disabled, None when there is no response.

Example: Query whether the button is enabled:

```
status = hub.get_button_control_status()
```

### set_device_address(address)

Description: The device address identifies and distinguishes each hub when multiple hubs are connected.

Parameters: address (int): user-defined address in the range 0x0000–0xFFFF.

Returns: bool: True when the device confirms the change, otherwise False.

Tip: creating a SmartUSBHub instance automatically reads the connected device's address. Before selecting devices by address, every device must be configured with a unique address.

Example: Set the device address to 0x0001:

```
hub.set_device_address(0x0001)
```

### get_device_address()

Description: Gets the device address.

Returns: int or None: the device address; None when there is no response.

Example: Query the device address:

```
device_address = hub.get_device_address()
```

### set_operate_mode(mode)

Description: Sets the device's operating mode.

Parameters: mode (int): operating mode (0 normal mode, 1 interlock mode).

Returns: bool: True if the command was applied successfully, otherwise False.

Note: in interlock mode, power can only be controlled via set_channel_power_interlock(); normal power commands have no effect. When a channel is powered off, its data connection is disconnected as well.

Example: Set the device to normal mode:

```
hub.set_operate_mode(0)
```

### get_operate_mode()

Description: Queries the device's current operating mode.

Returns: int or None: the current operating mode; None when there is no response.

Example: Query the device operating mode:

```
mode = hub.get_operate_mode()
```

### factory_reset()

Description: Resets the device to factory settings.

Returns: bool: True if the command was applied successfully, otherwise False.

Example:

```
hub.factory_reset()
```

### identify_device()

Description: Blinks the device's status LED rapidly to help locate it physically among multiple devices.

Returns: bool; True when the device confirms the command, otherwise False.

### set_device_alias(alias) and get_device_alias()

Description: Sets or reads a UTF-8 device alias.

Parameters: alias (str): UTF-8 alias, up to 15 bytes; an empty string clears it.

Returns: set_device_alias() returns bool; get_device_alias() returns the alias string, or an empty string when unset or unsupported by older firmware.

Example:

```
hub.set_device_alias("Rack A")
print(hub.get_device_alias())
```

### set_channel_name(channel, name) and get_channel_name(channel)

Description: Sets or reads a single channel's UTF-8 display name.

Parameters: channel (int): channel number. name (str): UTF-8 name, up to 15 bytes; an empty string restores the default CHn.

Returns: set_channel_name() returns bool; get_channel_name() returns the stored name or the default channel name.

### get_channel_names(*channels)

Description: Reads the display names of the specified channels.

Parameters: *channels (int): channels to read; all valid channels when omitted.

Returns: dict[int, str].

Example:

```
hub.set_channel_name(1, "DUT")
names = hub.get_channel_names()
```

### reboot_mcu()

Description: Requests an MCU reboot. The device disconnects shortly after acknowledging and usually needs to be reconnected.

Returns: bool; True when the device confirms the command, otherwise False.



NOTE: Stop all writes and firmware operations on downstream devices before rebooting

### reset_hub()

Description: Resets the USB data hub inside the device. Downstream USB devices disconnect and may re-enumerate; the MCU, the control serial port, and port power policies are unaffected.

Returns: bool; True when the command executes successfully, otherwise False.



NOTE
Note: This API applies only to models that declare the device.reset_hub capability. Unsupported devices raise FeatureNotSupportedError.

## Device Information and Discovery

### get_channels()

Description: Returns all valid channel numbers of the connected product; channel numbers start at 1.

Returns: tuple: available channels of the current device, e.g. (1, 2, 3, 4) or (1, 2, 3, 4, 5, 6, 7).

Example:

```
channels = hub.get_channels()
hub.set_channel_power(*channels, state=1)
```

### get_identity()

Description: Returns cached device identity information, including serial port path, serial number, address, alias, model, channel count, hardware/firmware versions, operating mode, and more. Field values are raw types and serialize directly to JSON; unlike get_device_info(), missing values remain None instead of "N/A".

Related: for declared capability, specification, and hub identity information, prefer get_capability(); use get_identity() for the MCU serial number, address, and versions. get_device_info() remains as a legacy summary/re-query interface and is no longer recommended as the ordinary read entry point.

Returns: dict: a dict of device identity information.

Example:

```
identity = hub.get_identity()
print(identity["model"], identity["serial"])
```

### get_product_type()

Description: Queries the product type ID of the connected device.

Returns: int or None: the product type ID; None when there is no response.

Example:

```
product_type = hub.get_product_type()
```

### get_capability(refresh=False)

Description: Returns the firmware's JSON capability declaration as a Python dict (an independent copy), containing the summary and all declared detail groups. By default it reads the cache taken at connection time; returns None when no firmware declaration exists and never fabricates fields for a model. Top-level capabilities may be plain booleans or dicts with a supported key; the firmware declaration structure is authoritative. Use json.dumps(caps, ensure_ascii=False) when JSON text is needed.

Parameters: refresh (bool): True re-queries the summary and all declared detail groups.

Returns: dict or None.

Example:

```
caps = hub.get_capability()
if caps:
    measurement = caps.get("measurement")
    supported = measurement is True or (
        isinstance(measurement, dict) and measurement.get("supported") is True
    )
    if supported:
        values = hub.get_channel_measurements()
```

### get_product_name()

Description: Queries the product name of the connected device, e.g. HBP_USB2_4CH or HBP_USB2_7CH_ADV.

Returns: str or None: the product name; None when there is no response.

Example:

```
product_name = hub.get_product_name()
```

### get_max_channels()

Description: Queries the device's maximum channel count. New code should usually prefer get_channels() for the list of actually available channels.

Returns: int or None: the maximum channel count; older firmware may not support this command, in which case None is returned.

Example:

```
max_channels = hub.get_max_channels()
channels = hub.get_channels()
```

### get_serial_no()

Description: Queries the device serial number.

Returns: str or None: the device serial number; may return "N/A" when the device does not provide one.

Example:

```
serial_no = hub.get_serial_no()
```

### get_firmware_version()

Description: Queries the device firmware version.

Returns: int or None: the firmware version; None when there is no response.

Example:

```
firmware_version = hub.get_firmware_version()
```

### get_firmware_version_major()

Description: Returns the cached firmware major version; queries the device first when necessary. Older firmware is treated as major version 1.

Returns: int or None.

### get_firmware_version_string()

Description: Formats the cached firmware version as a display string, e.g. V2.1.

Returns: str; "Unknown" when the version cannot be read.

### get_hardware_version()

Description: Queries the device hardware version.

Returns: int or None: the hardware version; None when there is no response.

Example:

```
hardware_version = hub.get_hardware_version()
```

## Event Callbacks

### register_callback(cmd, callback)

Description: Registers a user callback for a given command. The callback fires when the device replies to that command or proactively reports the corresponding event.

cmd may also be "disconnect": on an unexpected disconnect, a zero-argument callback is invoked by the receiving thread after port cleanup; an explicit disconnect() call does not trigger it. Pass None to unregister, and registering the same event again replaces the previous callback. Command-code callbacks receive the raw reply payload.

Parameters cmd (int or str): the command code to register; pass "disconnect" for the host-side disconnect event. callback (function): invoked when the command's ACK is received. A command-code callback takes two arguments, channel (int) for the channel number that triggered it and status (int) for the channel status value.

No return value.

Note: registerable command codes come from the SDK-exported CMD_* constants: all commands that wait for a device acknowledgment, plus the two proactive events CMD_ALERT_EVENT and CMD_DEBUG_LOG; cmd="disconnect" is a host-side event and uses no command code. Command codes outside this set log a warning and the callback is not registered. The callback signature should match the data structure returned by the device. Ordinary control and query commands use fixed-length protocol frames. The variable-length protocol is used for high-speed or streamed data transfer, such as the measurement stream; ordinary application code does not need to select a protocol version manually.

Register a button callback that fires when a button press is detected:

Example:

```
from smartusbhub import CMD_GET_CHANNEL_POWER_STATUS

def button_press_callback(channel, status):
    print("Button press detected on channel", channel, "with power status", status)

hub.register_callback(CMD_GET_CHANNEL_POWER_STATUS, button_press_callback)
```